

IDE

*Com Carinho pra meu Amigo
Michael Douglas e seus alunos
Herick Betin Tiburski*

EMBEDDED

HERICK BETIN TIBURSKI

Introdução

“Ah, mas você é bobo... sua IDE se chama literalmente IDE?”

Mano, eu gostei do nome. E sinceramente acho que faz sentido, porque ela é uma IDE para embarcados. Mas ao mesmo tempo ela traz uma proposta diferente: aqui você não programa tudo na mão, você programa eventos. Por exemplo, imagine um botão. Normalmente você teria que escrever todo o código de leitura do pino, lembrar de debounce, tratar estados, temporização e várias outras coisas. Aqui não. Aqui você só se preocupa com o que vai acontecer quando o botão for clicado. Nós trabalhamos da seguinte forma: existe uma função base que contém toda a lógica interna do componente e dentro dela existem eventos, como por exemplo `eventAoClicar()`. E é justamente nesses eventos que você trabalha. Isso traz muito mais facilidade na hora de desenvolver projetos. Chamamos isso de Paradigma Orientado a Eventos de Hardware (Programação Reativa).

Tá, mas por que eu pensei em fazer isso? Cara, basicamente porque quando eu era criança eu amava programar no Construct 2. Você criava um personagem e simplesmente dizia “quero que ele ande”, pronto, o software já fazia aquilo da melhor forma possível. Depois era só pensar na lógica: “ah, quero que quando clicar no personagem a vida diminua”. Era tudo muito visual, direto e fácil de manipular. Claro, não era exatamente igual ao que estamos fazendo aqui, mas a ideia era parecida: funções prontas, foco na criatividade e menos tempo brigando com implementação básica.

Mais tarde conheci o Sketchware através de um amigo meu que fazia hack de jogo pelo celular. E cara... aquilo era incrível. O aplicativo criava APK direto no celular. Mesmo sem nunca ter tido aula de programação eu conseguia desenvolver coisas sozinho. E isso vale tanto para o Sketchware quanto para o Construct: eu NUNCA parei para assistir tutorial. Claro, talvez hoje eu soubesse fazer algumas coisas de forma mais otimizada e perceberia várias gambiarras que eu fazia, mas funcionava. Eu sou aquele tipo de pessoa que odeia ler, mas ama escrever um livro.

Como eu já tinha uma base do Construct, não demorou muito para aprender o Sketchware. Desenvolvi um Instagram, um TikTok e até um “cassino honesto” KKKK, porque como eu nunca tinha estudado sistema de probabilidade, basicamente o jogo era “sim ou não”. E tudo isso sem tutorial. Bastava criatividade. Era isso que tornava o desenvolvimento possível.

Essa plataforma é basicamente um “Sketchware para embarcados”. E como para mim foi fácil aprender dessa forma, acredito que para você também possa

ser. Quando entrei na faculdade, na primeira aula de programação, eu já era um dos mais avançados da turma, até mais do que pessoas que tinham feito cursinho. E olha que eu nunca tinha realmente programado “na mão” antes. Sempre tive medo de código tradicional. Eu me sentia limitado, mesmo sabendo que ele era extremamente poderoso. Talvez porque existam possibilidades demais, não sei... pode ser viagem minha. Mas mesmo depois disso tudo eu ainda evitava programar escrevendo código puro. Eu odiava a ideia de colocar a mão em um monte de código. Só que na faculdade fui obrigado, e ironicamente comecei a gostar.

Essa plataforma não existe para substituir programação tradicional. Ela existe para ensinar lógica de programação antes da pessoa entrar no “bruto” e acabar se perdendo. Na vida não adianta querer começar pelo mais difícil. Muitas vezes isso só desgasta, desmotiva e até limita a criatividade. Eu acredito que criatividade nasce do simples e do direto, não do complicado e confuso. E isso não anula o fato de você aprender programação tradicional depois.

Enfim, neste livro eu vou apenas mostrar a plataforma e ensinar como utilizá-la. O resto eu quero deixar com vocês. Espero de verdade que vocês gostem. Talvez eu deixe a plataforma open source no futuro para que outras pessoas possam estudar a ideia e até criar novas plataformas baseadas nesse conceito. E desculpa pela introdução gigante, mas se você leu tudo isso até aqui, cara... você é incrível. Espero que goste desse meu trabalho. Estão sendo noites e noites viradas desenvolvendo isso, e espero que essa plataforma possa contribuir tanto para iniciantes em embarcados quanto para profissionais que queiram experimentar essa modelagem. A ideia é simples: código simples, mas com uma modelagem moderna e poderosa de componentes. Mas é isso... vamos lá.

Primeiro contato

Vamos lá, o primeiro contato com a plataforma é o que deixa a pessoa mais animada. Cara, para mim, não tendo anúncio, já é a melhor IDE do mundo. Brincadeira essa parte. Mas enfim, ela possui uma interface clean, pode ser que mude algumas coisas, porém a essência e esse livro nunca vai se anular pois a lógica sempre vai ser a mesma, nessa versão Não temos muitas coisas, porém vou explicar cada uma pra você.



Essa é nossa barra de ferramentas — não sei ao certo como chamar isso. Aqui temos um papel em branco, que representa “Novo Projeto”, justamente por estar em branco. Depois temos uma pasta aberta com um papel, que serve para abrir um projeto já existente. Em seguida temos o clássico disquete, universalmente conhecido como salvar projeto.

E temos tbm UNDO e REDO pra dar basicamente Ctrl+z e +Y

Logo depois temos duas ferramentas, que no RARS, um simulador RISC-V, são usadas para fazer o build do código. Aqui usamos exatamente para a mesma função. Depois temos o botão de play, responsável por iniciar a simulação do projeto.

Em seguida temos um leitor de DVD com um DVD. Cara... eu sinceramente não sabia o que colocar para representar o flash da ESP, então ficou isso mesmo. Talvez vocês tenham que me dar um desconto, porque o Windows tem poucas opções de ícones.

Logo depois temos um papel branco com uma seta. Esse botão serve para exportar o projeto. Porém, no caso, ele exporta o PNG do circuito pronto para fazer a PCB em máquina a laser. Eu sei que talvez não esteja tão intuitivo, mas como falei antes, as opções de ícones eram bem limitadas.

Ao lado temos um monitor, que ao clicar permite configurar o compilador. Também não combina tanto assim, mas prometo que no futuro vamos melhorar os ícones e vocês ainda vão lembrar desse livro pensando: “caraca... era assim antigamente”.

Depois temos a lixeira, que limpa completamente o workspace. Logo após vem uma espécie de mesa de configuração, que serve para modelar componentes. Então, caso a plataforma não tenha o componente que você precisa, você mesmo pode criar o seu.

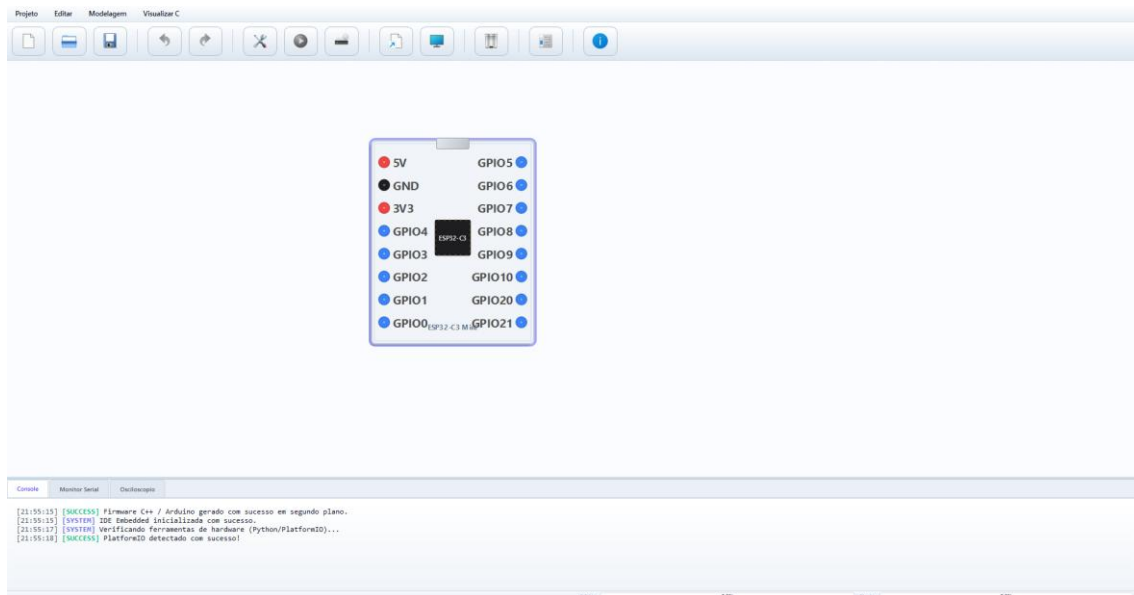
E por último temos o “i”, onde você encontra informações da versão da IDE, documentação, acesso ao livro e outras informações importantes.

Atalhos e funções

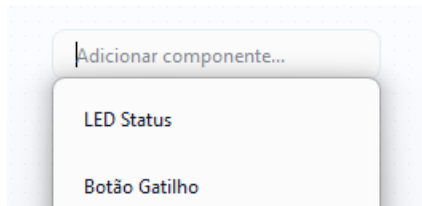
- Ao clicar 2 vezes na área de trabalho, irá aparecer o menu de componentes.
- Ao pressionar DEL, o componente selecionado será deletado.
- Ao clicar com o botão direito no componente, os eventos serão exibidos.
- Ao entrar em um evento e clicar 2 vezes na área de eventos, aparecerá o menu de código.
- Dando 2 cliques em um bloco de código, ele será deletado.
- Quando estiver fazendo uma trilha, clicar com o botão esquerdo prende a trilha para conseguir movê-la para outro lado.
- Caso erre a trilha, clique com o botão direito do mouse para desfazer a trilha e soltá-la do ponteiro.
- Caso veja trilhas como VCC que já podem se conectar, basta clicar nela para prender a trilha atual na que já está conectada ao VCC.

Primeiro Exemplo de execução

Agora vamos aprender a usar.



Você percebeu que não tem nenhum botão de componente, né? Então faça assim: dê dois cliques no workspace e vai aparecer uma barra de busca.



Escreva “botão” e clique duas vezes nele.

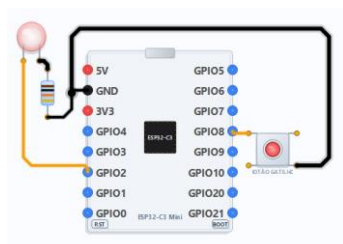
Pronto, agora você tem um botão.

Agora, em vez de clicar novamente no workspace, apenas escreva “LED” e clique nela.

Pronto. Agora conecte ela na ESP. Como o LED usa um resistor, dê dois cliques ou escreva “resistor” e coloque um.

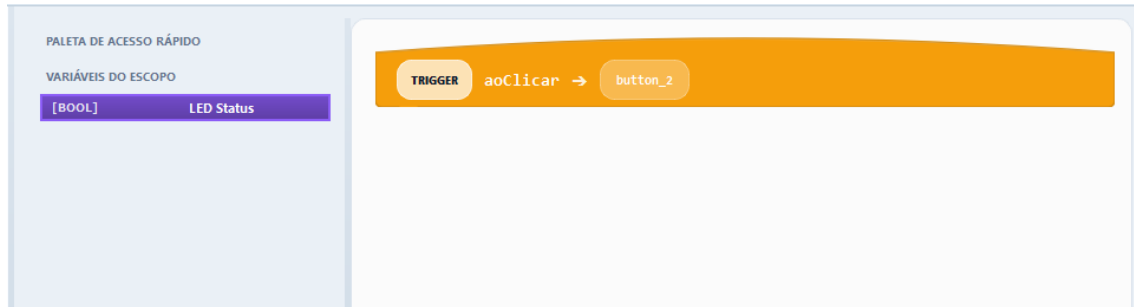
Se quiser girar um componente, clique com o botão direito e selecione “Girar 90°”. Depois, faça as conexões.

Para mudar a cor das trilhas, clique com o botão direito e “Alternar Cor da Trilha”.



Agora vá no botão e clique com o botão direito. Vai aparecer um menu com “Eventos”. Clique em “Ao clicar”.

Pronto, agora você abriu os eventos. Aqui você irá definir o que vai acontecer ao clicar no botão.



Nos eventos, clique duas vezes e escreva “Ação”, depois selecione.

Esse bloco basicamente faz quase tudo, porém vamos apenas ligar o LED, então mantenha o padrão.

Olhando na esquerda, temos as variáveis e uma delas é o LED. Puxe ela e coloque dentro do espaço de escrita.

Pronto.



Agora clique nas duas ferramentas para fazer a build.

Depois, clique Simular agora.

ele vai liberar o botão de simulação. Clique nele e o osciloscópio vai aparecer para você.



Clique no botão e o LED vai ligar.

Porém, note uma coisa: o LED não desliga. Isso acontece porque ainda não configuramos isso.

Então pare a simulação e clique com o botão direito no LED selecione o evento AoLigar. Escreva “Aguardar”, coloque 100 ms e depois adicione outra ação, coloque o LED e selecione LOW clicando no HIGH.

Pronto. Agora faça a build novamente, rode a simulação e pronto: temos uma simulação de ligar e desligar o botão.

Exportar PCB para PNG

Agora vamos exportar para PNG para conseguir fazer o circuito na PCB.

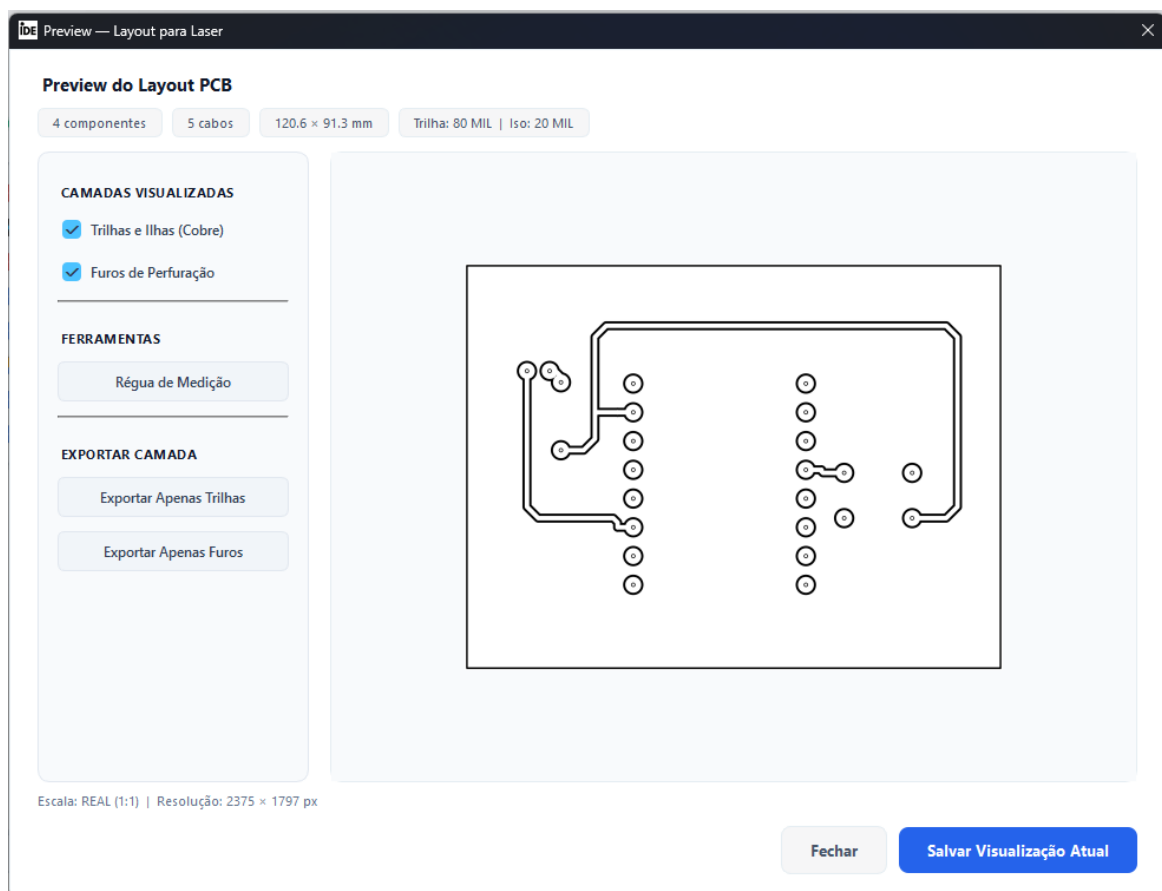


Clique no botão onde tem um papel com uma seta.

Agora clique em “Preview”.

Pronto. Agora ele irá gerar uma visualização pronta do circuito para você usar na impressora a laser para fazer a PCB.

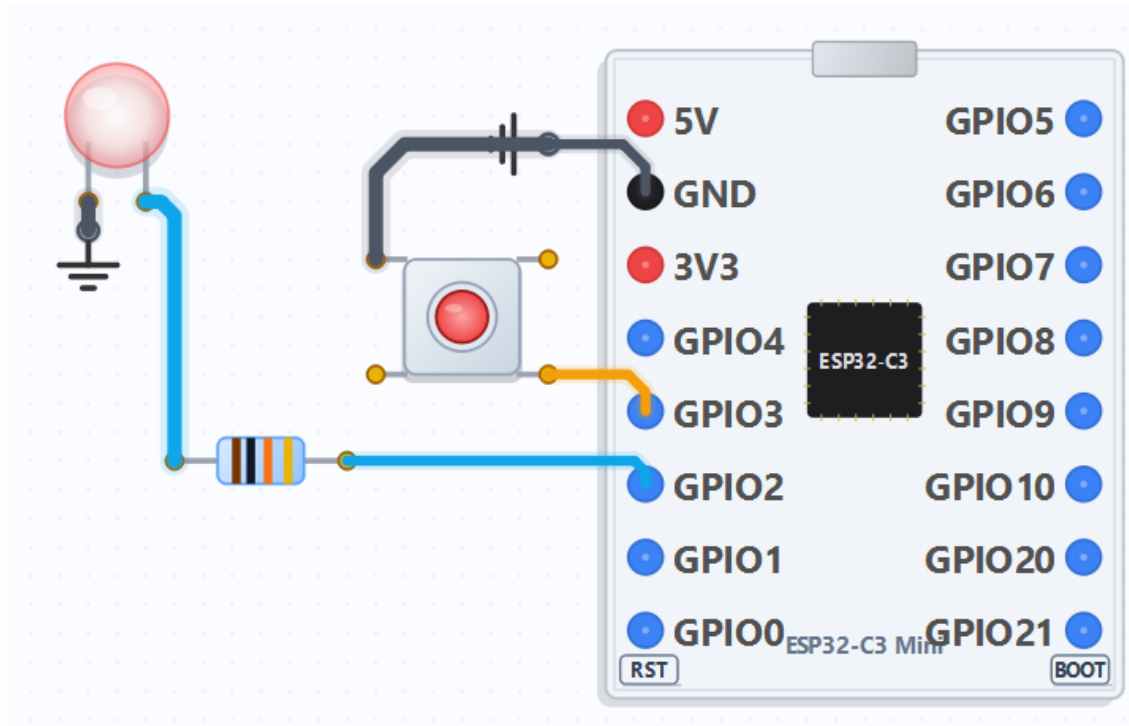
Ficaria basicamente assim:



Com isso, você já consegue imaginar o que é possível fazer. E saiba que ainda tem muita coisa que dá pra criar. Vamos começar pelo básico aqui. O que fizemos até agora já permite você fazer várias coisas, como ligar buzzer e outras funcionalidades.

Agora vamos aprender a salvar coisas e ler dados usando EEPROM. Vamos usar o mesmo circuito e depois começaremos a avançar para novos projetos. Mas, antes de eu te ensinar, quero que você tente fazer sozinho.

Antes disso, uma otimização do sistema seria usar o componente GND. Basicamente, colocando ele no GND, você consegue fazer com que tudo que não está isolado seja considerado GND. Isso evita ficar toda hora conectando tudo com trilhas no GND. O GND funciona como um ponto de referência e retorno da corrente no circuito



Analisando, você percebe que ele retira os pontos que seriam o GND. Isso faz o GND “vazar” pela placa. Basicamente, em vez de precisar ligar tudo com várias trilhas de GND, a própria área de cobre vira um caminho compartilhado de aterramento. Isso é parecido com um ground plane, onde uma grande área da placa inteira funciona como GND para os componentes.

Além de deixar o circuito mais limpo visualmente, isso também ajuda no retorno de corrente, reduz ruído e evita um monte de trilhas desnecessárias. Em PCBs reais, esse conceito é muito usado justamente porque o GND vira uma referência espalhada pela placa inteira.

Gravando e lendo na EEPROM

Agora sim, vamos lá. Vamos fazer 2 exemplos básicos, só que antes vou pedir pra você tentar sozinho. Não vale ver o resultado antes. Se fizer isso, você tá enganando a si mesmo. Lembra: quando você fuça, aprende até mais do que quando alguém te ensina. É errando que se aprende, e é errando sozinho, com atitudes suas.

Bora lá:

- Quero que grave o estado do LED para que, quando ligar novamente, ele volte ao estado anterior. Para ligar e desligar o LED na simulação, clique em reset.
- Quero que toda vez que ligar, leia o estado do LED e mostre no Serial Print.

Antes de fazer isso, vamos mudar a lógica do botão. Exclua os blocos do botão e também do LED. No botão, faça algo tipo: se já estiver HIGH, fica LOW; se estiver LOW, fica HIGH. Simples. Não vou mostrar como vai ficar, faz aí. Agora sim, pode implementar.

A EEPROM é uma memória não volátil, ou seja, ela mantém os dados mesmo depois de desligar a placa. Ela é muito usada justamente para salvar estados, configurações e variáveis persistentes.

Resposta da implementação

Então, se você só pulou essa parte, precisa rever seus conceitos, porque eu sei que sou meio chato aqui, mas se você não tentar sozinho, nunca vai aprender. Você só vai fazer um Ctrl+C Ctrl+V do que eu falei e você aceitou. Você precisa explorar, descobrir como funciona e criar sua própria lógica mental de programação.

Então vamos lá, essa é a resposta de como tem que ficar a nova lógica de ligar e desligar:



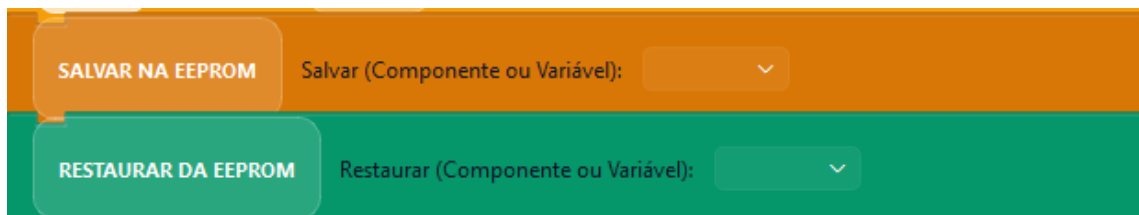
Podemos analisar que ao LED tiver HIGH vai ficar LOW e vice-versa bem fácil

Agora, como vai funcionar para salvarmos e restaurarmos? Simples: usamos o bloco **“Salvar estado do componente/variável”** e selecionamos o LED. Isso também funciona para variáveis, tá bom?

Depois, no setup, você coloca o bloco **“Restaurar”** e escolhe o LED.

O que ele faz? Basicamente, ele salva o estado para você e, como ele sabe exatamente onde armazenou na memória, consegue recuperar esse valor depois. Assim, ele restaura automaticamente o estado salvo, tornando sua vida muito mais fácil.

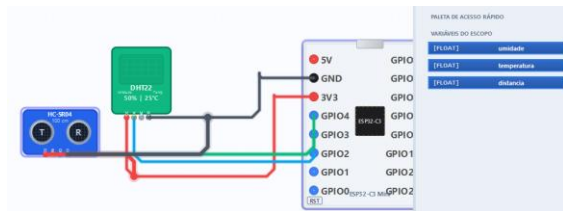
Os blocos são estes:



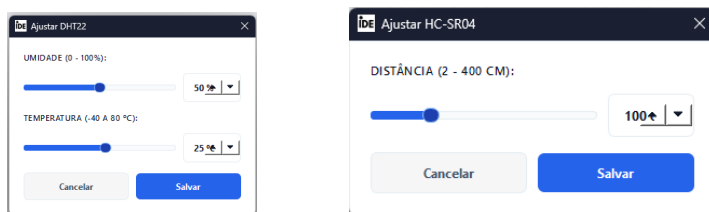
Sensores

Como funciona sensores

Temos alguns sensores padrões por exemplo HC-SR04, DHT22 entre outros, pra utilizar é mais fácil que tudo, acho que é o menos trabalhoso de todos, simplesmente coloque os componentes e pronto você terá as variáveis dos valores na aba variáveis do escopo:



Dois cliques nos sensores aparece pra vc mudar os parâmetros simulados



Exercício I

Agora vamos fazer um projeto grande baseado em um projeto meu, o Óculos SoundVision. Basicamente, ele consiste em 3 sensores HC-SR04, 3 buzzers e uma ESP32-C3 Mini. A bateria vamos aprender a implementar depois.

Então vamos lá: o funcionamento dele é simples. Quanto mais próximo o objeto estiver, mais rápido o buzzer vai apitar, de acordo com o sensor de distância que estiver detectando. Simples e fácil.

Vamos implementar. Como fizemos antes, tente fazer primeiro sozinho; depois vamos fazer juntos.

Segue o cálculo necessário:

$\text{distancia} * 10$

Basicamente, isso serve para não ficar apitando igual louco quando estiver, por exemplo, a uns 50 cm de distância.

Lembrando que precisa existir:

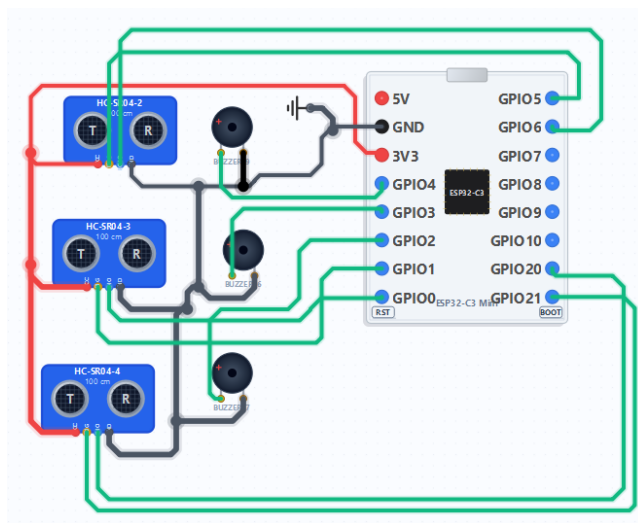
- o tempo entre cada bip;
- e também o tempo de duração do bip.

Resposta do projeto

Vamos lá, uma observação: este projeto é de minha autoria e é 100% open source, criado para estudos e aprimoramento de dispositivos de auxílio para pessoas com deficiência visual. A proposta dele é trazer uma noção espacial para o usuário, ajudando na percepção de obstáculos ao redor.

Quero muito que vocês olhem para essa ideia e também se inspirem a desenvolver novas tecnologias, principalmente na área de tecnologia assistiva. É uma sensação incrível quando você vê alguém feliz porque uma criação sua conseguiu ajudar de verdade no dia a dia dela.

Mas enfim, sem enrolação, vamos entender como projetar esse sistema.



Primeiro, defini os pinos de cada sensor e de cada buzzer. Como o software já faz os cálculos automaticamente para nós, as variáveis já ficam prontas na aba de eventos. Então o que realmente precisamos pensar é na lógica de funcionamento.

A primeira pergunta é: quando queremos que a pessoa comece a ouvir os bipes?

No meu caso, imaginei começar a detectar obstáculos a partir de 3 metros. Isso porque nessa distância ainda temos uma boa precisão do sensor e tempo suficiente para prever obstáculos, até mesmo os mais baixos.

É importante lembrar que o HC-SR04 funciona em formato de cone, porque o ultrassom se espalha dessa forma. Segue um exemplo:



Pensando nisso, podemos criar tabelas de distância e analisar qual comportamento fica melhor para o usuário.

Agora vamos implementar.

Podemos começar com uma lógica simples: se a distância for menor que 300 cm, o buzzer começa a apitar.

Tudo isso será colocado dentro do evento de cada sensor. Claro, também poderíamos fazer no loop(), mas dessa forma o projeto fica mais organizado e fácil de entender.

Uma observação importante: para sistemas desse tipo, Rust é uma linguagem muito interessante para embarcados, principalmente por facilitar a criação de sistemas assíncronos modernos, seguros e eficientes. Frameworks como o [Embassy](#) permitem trabalhar com múltiplas tarefas ao mesmo tempo de forma organizada, como leitura de sensores, controle dos buzzers e gerenciamento de bateria.

Beleza, já definimos quando o buzzer deve apitar. Agora vem a próxima parte: como fazer ele apitar na velocidade certa?

A lógica é bem simples:

- Ligamos o buzzer.
- Esperamos um pequeno tempo, por exemplo 50 ms.
- Desligamos o buzzer.
- Depois adicionamos outro delay baseado na distância detectada.

Assim, quanto mais próximo o objeto estiver, menor será o delay e mais rápido o buzzer irá apitar.

Basicamente é isso.

Nesse livro vamos reutilizar bastante esse código e ir evoluindo-o aos poucos, adicionando melhorias e novas funções. Então recomendo salvar agora clicando no disquete para não perder o projeto.

Acabou que eu perdi 2 dias de trabalho. Inventei de pedir para uma IA corrigir um erro e acabei perdendo tudo o que tinha feito. Nesses 2 dias eu tinha feito mudanças grandes, como a integração com RUST e a implementação da flash real. Porém, tudo o que ensinei até agora vai continuar igual, então podem seguir normalmente. Só estou comentando isso para vocês saberem mesmo. E uma dica: tomem cuidado ao usar IA em projetos importantes e usem o GitHub para não perderem as versões de cada atualização do projeto.

Dúvidas do projeto

Como fazer cálculos matemáticos?

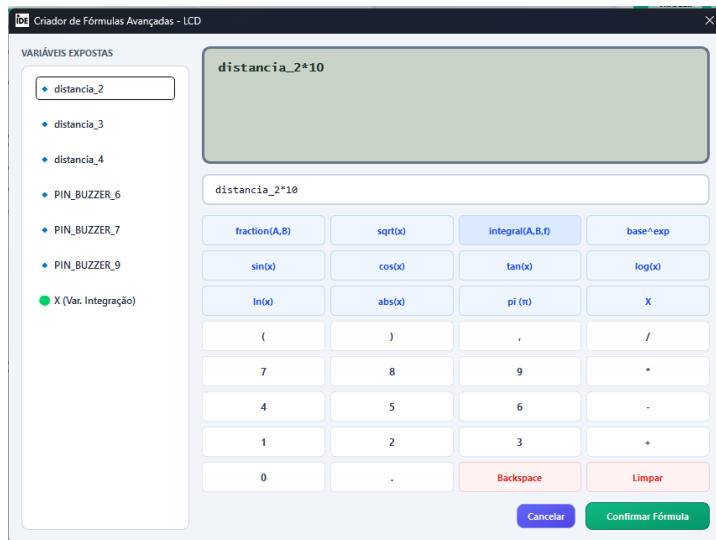
Adicione o bloco **Matemática** e selecione **Multiplicação**.

Depois, coloque o nome da variável que vai receber o resultado. Em seguida, arraste a variável `distancia_e` e, no outro campo, coloque 10.

Ficaria assim:



“Ah, mas assim só faz fórmula simples”, Clique `f(x)`:



Maneiro, né? Inspirada nas melhores calculadoras do mercado — não vou falar o nome porque não me patrocinam. Mas, enfim, é algo incrível para cálculos complexos.

Como simular as distâncias?

Clique 2 vezes no sensor e escolha a distância.

Como fazer aparecer a distância no Serial Print?

Clique 2 vezes no evento, escolha **ESCREVER** e coloque a variável `distancia` que você precisa analisar.

Você também pode colocar algo como:
Distância Sensor 1:

com a opção **nova linha**. Depois, adicione outros blocos **ESCREVER** com a opção **mesma linha** e coloque a variável `distancia_1`.